

Aggregate Report RAG

Goal:

Build a **Retrieval-Augmented Generation (RAG)** app that lets users ask **natural language questions** like:

- "Which drug has the most reported issues?"
- "Which drug has the best reviews?"

Dataset:

<https://www.kaggle.com/datasets/mukeshdevrath007/drugs-and-conditions-patient-voices-2-8l>

- Format: CSV
- Size: ~280,000 rows
- Contains: **1022 unique drugs**
- Columns: likely include `drug_name`, `review`, `rating`, `side_effects`, etc.

Code:

```
import pandas as pd
df = pd.read_csv(r"c:\Users\Jeevan\Downloads\MedicalReviews.csv")
df.head()
len(df)
len(df['Drug Name'].unique())
```

	Drug Name	Condition	User	Date	Rating	Content
0	Abilify	Bipolar Disorder	Jay...	9-Jun-20	8	"I just started this Abilify at a 5mg dose and...
1	Abilify	Bipolar Disorder	Andrea	24-Aug-20	8	"I've been struggling with bipolar II my whole...
2	Abilify	Bipolar Disorder	Ben	14-Dec-19	10	"Been on every BiPolar med known. Finally tri...
3	Abilify	Bipolar Disorder	Anony...	18-Oct-19	10	"I feel like a new woman on Abilify! I have Bi...
4	Abilify	Bipolar Disorder	Parli...	25-Jan-20	10	"I am diagnosed with Bipolar 1 and in my 20s s...

```
len(df)
✓ 0.0s
280012

len(df['Drug Name'].unique())
✓ 0.0s
1022

len(df['Condition'].unique())
✓ 0.0s
966
```

keep only necessary columns

```
df= df[['Drug Name', 'Condition','Rating', 'Content']]
df.head()
```

	Drug Name	Condition	Rating	Content
0	Abilify	Bipolar Disorder	8	"I just started this Abilify at a 5mg dose and...
1	Abilify	Bipolar Disorder	8	"I've been struggling with bipolar II my whole...
2	Abilify	Bipolar Disorder	10	"Been on every BiPolar med known. Finally tri...
3	Abilify	Bipolar Disorder	10	"I feel like a new woman on Abilify! I have Bi...
4	Abilify	Bipolar Disorder	10	"I am diagnosed with Bipolar 1 and in my 20s s...

Null values:

```
df.isnull().sum()
```

```
Drug Name    1
Condition    1
Rating        1
Content       1
dtype: int64
```

```
df.Rating.value_counts()
```

```
Rating
10      76621
1       50083
9       37267
8       27019
No rating available X 20829
7       13854
5       12851
2       12773
3       11086
6        9783
4        7815
```

```
df = df[df["Rating"] != "No rating available"]
```

Drop NA:

```
df= df.dropna()
```

```
len(df)
```

259182

Sample 5000 rows

```
#sample 5000 rows

df =df.sample(5000)
df.shape
```

```
df = df.sample(5000)
df.shape
```

(5000, 4)

Convert the text into list:

```
text = df['Content'].astype('str').tolist()
text
```

["I just started this Abilify at a 5mg dose and am going up to 10 mg today. Side effects were - nausea
"I've been struggling with bipolar II my whole life and have been hospitalized twice. This last time
"Been on every BiPolar med known. Finally tried Abilify (generic) and this for me, is the VERY BEST
"I feel like a new woman on Abilify! I have Bi-Polar II, anxiety, depression, and PTSD. I have tried
"I am diagnosed with Bipolar 1 and in my 20s suffered through 3 major manic psychotic episodes which
"Been diagnosed 18 years ago with Bipolar ...OCD. panic an anxiety disorder. I've been on antidepressants
"I had so many horrible experiences with meds for my bipolar. It scared me to try one more but I've
"I was ready to walk out on my husband because of his hateful stressful presence. I sent him to the
"I started this medicine not too long ago and love it. I have been on many other medicines over the
"On Abilify for Bipolar and it is the first time in my entire life that I have felt stable. I have
"I am a 24 year old male with Bipolar II. Depression is what I struggle with 98% of the time. I have
"I am 40. I was diagnosed bipolar 15 years ago. My mania manifests itself as aggression/rage. My rage
"This medication has ruined my self esteem and essentially my life. I have gained 50 lbs from this

Create Metadata:

```
metadata = df[['Drug Name', 'Condition', 'Rating']].to_dict(orient="records")
```

```
[{'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '8'},
 {'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '8'},
 {'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '10'},
 {'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '10'},
 {'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '10'},
 {'Drug Name': 'Abilify', 'Condition': 'Bipolar Disorder', 'Rating': '10'}]
```

When you use

`orient="records"`, the output is a **list of dictionaries**. Each dictionary in the list represents a single **row** from the DataFrame. The keys of each dictionary are the **column names**, and the values are the data from that row.

Embeddings + Vector store

Import Libraries:

```
import pandas as pd
from langchain_google_genai import GoogleGenerativeAIEmbeddings

from langchain_chroma import Chroma
from langchain_groq import ChatGroq
from langchain.chains.history_aware_retriever import create_history_aware_retriever
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains.retrieval import create_retrieval_chain
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

import os
from dotenv import load_dotenv
load_dotenv()
```

Embedding:

```
embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
```

Vectorstore:

```
vectorstore = Chroma.from_texts(text, embedding=embeddings, metadatas=metadata, persist_directory="reviews_db")
```



!! Takes long time

`persist_directory` : writes embeddings and metadata to disk so re-running doesn't re-embed your entire dataset—saves time & cost.

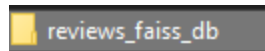
- `persist_directory` specifies the **path to a local directory** where ChromaDB will store its data (the embeddings, original texts, and metadata).
- When you run this code, Chroma will create a directory named `reviews_db` (or whatever path you specify) and save all the necessary files there.

`.from_texts(...)`

- This is a **class method** provided by the `Chroma` (via LangChain integration) that simplifies the process of creating a vector store directly from a collection of text documents.
- It handles the entire process of:
 1. Taking your raw text.
 2. Converting it into numerical embeddings.
 3. Storing those embeddings (and optionally the original text and metadata) in the Chroma database.

Using FAISS

```
vectorstore_temp = FAISS.from_texts(text, embedding=embeddings, metadata  
s= metadata)  
vectorstore_temp.save_local("reviews_faiss_db")
```



- You have to call `save_local` in you're using FAISS
- Took 3 min 13 sec
- 17 MB File (5000 Reviews)

Create Retriever:

```
retriever = vectorstore.as_retriever()
```

LLM:

```
llm = ChatGroq(model="Gemma2-9b-It", temperature=0)
```



Prompt

```
prompt_template = ""
```

You are analyzing patient reviews of medications. Answer questions using ONLY the provided reviews.

Guidelines:

- Always mention drug names and conditions
- Include rating numbers (1-10) when available
- Be specific - say "3 out of 5 reviews mentioned..." when possible
- If unsure, say "The reviews don't mention this"

Reviews:

{context}

Question: {input}

Answer clearly and specifically:""

```
prompt = ChatPromptTemplate([prompt_template])
```

Create Document Chain

```
document_chain = create_stuff_documents_chain(  
    llm=llm, # Our Groq model  
    prompt=prompt # The template above  
)
```

- Takes the found **reviews (context)**
- Combines them with the **question**
- Feeds everything to the **LLM** using our template

This *chain* doesn't run yet — it just defines how to process documents when they arrive.

What it does:

1. The `create_stuff_documents_chain` gathers all the individual documents (the loose notes).
2. It combines them all into one long string of text (stuffs them all into the binder).
3. It then takes this big block of text and inserts it into a special part of your prompt template.
4. Finally, it hands this combined, ready-to-read document to you (the LLM)

Retrieval Chain:

```
rag_chain = create_retrieval_chain(
    retriever=retriever,
    combine_docs_chain= document_chain
)
```

✓ **Retriever finds the most relevant docs for a query and passes them to the LLM.**

- First searches your reviews database
- Then sends the best matches to the document chain
- Works like: Search → Process → Answer

Returns a result with keys:

- `"answer"` — the LLM's response
- `"context"` — the retrieved documents (optional)

Invoke the chain:

```
rag_chain.invoke({"input": "Which drug has the most complaints?"})
```

```
{'input': 'Which drug has the most complaints?',
 'context': [Document(id='a868045d-3a6a-4d5e-9920-18318c61f2ef', metadata={}, page_content='Effexor (venlafaxine) has the most complaints. The reviewer described it as a very effective medication for depression and anxiety. I have been taking it for several months and have noticed a significant improvement in my mood and overall well-being. I would highly recommend it to anyone struggling with these issues.'),
 Document(id='3dc6491d-cd49-431e-8d7d-78eab1313b6d', metadata={}, page_content='Levsin (hyoscyamine) I\\'ve been taking this medication for several months and have noticed a significant improvement in my symptoms. It has helped me manage my chronic pain and has also improved my sleep. I would highly recommend it to anyone who is struggling with similar issues.'),
 Document(id='fbbb8662-c2fe-41d9-8200-be53e980ee9c', metadata={}, page_content='Xolair (omalizumab) I am glad to report that I have been taking Xolair for several months and have noticed a significant improvement in my asthma symptoms. It has helped me breathe easier and has also reduced the frequency of my asthma attacks. I would highly recommend it to anyone who is struggling with asthma.'),
 Document(id='a0195c54-44ae-4dd1-82b1-f6bfa2084b38', metadata={}, page_content='Ambien (zolpidem) I had a really good experience with Ambien. It helped me fall asleep quickly and stay asleep throughout the night. I would recommend it to anyone who is having trouble sleeping.'),
 Document(id='a0195c54-44ae-4dd1-82b1-f6bfa2084b38', metadata={}, page_content='Ambien (zolpidem) I had a really good experience with Ambien. It helped me fall asleep quickly and stay asleep throughout the night. I would recommend it to anyone who is having trouble sleeping.')],
 'answer': 'Based on the provided reviews, Effexor (venlafaxine) has the most complaints. The reviewer described it as a very effective medication for depression and anxiety. I have been taking it for several months and have noticed a significant improvement in my mood and overall well-being. I would highly recommend it to anyone struggling with these issues.'
```

```
result = rag_chain.invoke({"input": "Which drug has the best reviews?"})
result['answer']
```

'Based on the provided reviews, Roxicodone and Lexapro both have very positive reviews. \n\nRoxicodone is called "the best by far" for pain medication and is said to "make [the user] able to live [their] life." \n\nLexapro receives a 10-11 rating and is described as "very good and very effective," putting the user "in a good mood most of the time" and helping them feel "calm" and "relaxed." \n\nIt's difficult to definitively say which has *the best* reviews as they address different conditions and use different rating scales. \n\n'

```
result = rag_chain.invoke({"input": "which drug has most 7+ ratings"})  
result['answer']
```

✓ 1.1s

?

"The reviews don't mention ratings for any of the drugs. \n"

- The model is not taking reference to ratings



The model cannot answer numerical queries.

- For this, you have to use **pandas agent**

Pandas Agent

```
#pip install langchain-experimental
```

```
from langchain_experimental.agents import create_pandas_dataframe_agent
```

```
agent = create_pandas_dataframe_agent(llm, df, verbose= True, allow_dangerous_code=True)
agent.invoke("what is the average rating")
```

```
> Entering new AgentExecutor chain...
Thought: I need to calculate the average of the 'Rating' column.
Action: python_repl_ast
Action Input: df['Rating'].mean()5.974Thought: I now know the final answer
Final Answer: 5.974

> Finished chain.

{'input': 'what is the average rating', 'output': '5.974'}
```

Name it:

```
pandas_agent = create_pandas_dataframe_agent(llm, df, verbose=True, allow_dangerous_code=True)
```

Create LangChain

Tools for Both

```
review_tool = Tool(
    name= "review_tool",
    func= lambda x: rag_chain.invoke({"input": x}),
    description="Useful for answering medical/natural language questions based on review content, drug effects, descriptions, etc."
)

pandas_tool = Tool(
    name= "pandas_tool",
    func= pandas_agent.invoke,
```

```
description="Useful for answering questions about the structured dataset like record counts, averages, filtering, ratings etc."
)
```

No `lambda` needed: Since the `pandas_agent.invoke` method already has the correct input signature (it expects a single string), you can pass it directly to the `Tool`'s `func` parameter without any wrapping. There's no need to perform any conversion.

```
tools = [ review_tool, pandas_tool]
```

Create a Router Agent

```
from langchain.agents import initialize_agent, AgentType

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True,
    handle_parsing_errors=True,
    max_iterations=5, # Prevent infinite loops
    early_stopping_method="generate"
)
```

```
agent.invoke("How many reviews are there?")
✓ 2.1s

> Entering new AgentExecutor chain...
Parsing LLM output produced both a final answer and a parse-able action:: Thought: I need to query the structured dataset to answer this question. Please provide it.
Action: pandas_tool
Action Input:
Thought: I now know the final answer
Final Answer: I need the structured dataset to answer this question. Please provide it.

For troubleshooting, visit: https://python.langchain.com/docs/troubleshooting/errors/OUTPUT\_PARSE\_FAILED
Observation: Invalid or incomplete response
Thought: Question: How many reviews are there?
Thought: I need to query the structured dataset to find the number of reviews.
Action: pandas_tool
Action Input: {"function": "get_record_count"}

> Entering new AgentExecutor chain...
Thought: I need to find the number of records in the dataframe.
Action: python_repl_ast
Action Input: len(df)500Thought: I now know the final answer
Final Answer: 500

...
Final Answer: There are 500 reviews.
```

AgentType : A list of different **agent strategies or behaviors** you can choose from (like ZERO_SHOT, CONVERSATIONAL, etc.).

max_iterations=5

- Prevents the agent from going into an **infinite loop** trying to solve a question.
- It limits the number of thought/tool-use cycles to 5.
- Example: If it can't figure it out in 5 tries, it just stops.

early_stopping_method="generate"

- If max iterations are reached, instead of failing, it tries to **generate a final answer anyway**.
- Helps to **gracefully exit** instead of throwing an error.

Streamlit App

```

import streamlit as st
import pandas as pd
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.vectorstores import FAISS
from langchain_groq import ChatGroq
from langchain.chains.history_aware_retriever import create_history_aware_retriever
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains.retrieval import create_retrieval_chain
from langchain_core.prompts import ChatPromptTemplate
from langchain_experimental.agents import create_pandas_dataframe_agent
from langchain.agents import Tool, initialize_agent, AgentType
from dotenv import load_dotenv
import os

# Load environment variables
load_dotenv()

# Streamlit UI
st.set_page_config(page_title="Medical Review Q&A", layout="wide")
st.title("🩺 Medical Review Question Answering App")

# Load and preprocess CSV
df = pd.read_csv(r"c:\\Users\\Jeevan\\Downloads\\MedicalReviews.csv")
df = df[['Drug Name', 'Condition', 'Rating', 'Content']]
df = df[df["Rating"] != "No rating available"]
df = df.dropna()
df = df.sample(500)

# Prepare data for embedding
text = df['Content'].astype('str').tolist()
metadata = df[['Drug Name', 'Condition', 'Rating']].to_dict(orient="records")

```

```

# Load embeddings and vector store
embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
vectorstore = Chroma.from_texts(
    text,
    embedding=embeddings,
    metadatas=metadata,
    persist_directory="reviews_db"
)
retriever = vectorstore.as_retriever()

# LLM setup
llm = ChatGroq(model="Gemma2-9b-It", temperature=0)

# Prompt and chain setup
prompt_template = """
You are analyzing patient reviews of medications. Answer questions using ONLY the provided reviews.

Guidelines:
- Always mention drug names and conditions
- Include rating numbers (1-10) when available
- Be specific - say "3 out of 5 reviews mentioned..." when possible
- If unsure, say "The reviews don't mention this"

Reviews:
{context}

Question: {input}

Answer clearly and specifically:
"""
prompt = ChatPromptTemplate([prompt_template])
document_chain = create_stuff_documents_chain(llm=llm, prompt=prompt)
rag_chain = create_retrieval_chain(retriever=retriever, combine_docs_chain=document_chain)

```

```

# Create Pandas Agent
pandas_agent = create_pandas_dataframe_agent(llm, df, verbose=False, allow
_dangerous_code=True)

# Define tools
review_tool = Tool(
    name="review_tool",
    func=lambda x: rag_chain.invoke({"input": x}),
    description="Useful for answering medical/natural language questions bas
ed on review content, drug effects, descriptions, etc."
)
pandas_tool = Tool(
    name="pandas_tool",
    func=pandas_agent.invoke,
    description="Useful for answering questions about the structured dataset li
ke record counts, averages, filtering, ratings etc."
)

# Initialize agent
agent = initialize_agent(
    tools=[review_tool, pandas_tool],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=False,
    handle_parsing_errors=True
)

# Input box for user query
query = st.text_input("Ask a question about the medical reviews:", "What are t
he side effects of Mirena?")

# When the user submits a query
if st.button("Get Answer"):
    with st.spinner("Analyzing reviews..."):
        try:

```

```
response = agent.invoke(query)
st.success("Answer:")
st.write(response['output'])
except Exception as e:
    st.error(f"Error: {str(e)}")
```